**Pimpri Chinchwad Education Trust's**
**Pimpri Chinchwad College of Engineering & Research Ravet, Pune**
An Autonomous Institute | NBA Accredited (4 UG Programs) |
NAAC A++ Accredited | ISO 21001:2018 Certified
**IQAC PCCOER**

## DEPARTMENT OF COMPUTER ENGINEERING

# Lab Manual

# PCC304COM: Artificial Intelligence Laboratory

**Third Year of Computer Engineering**

**(2024 Course)**

(Prepared by – Mrs. Priyadarshani Doke)

| | | |
|---|---|---|
| | **Pimpri Chinchwad Education Trust's**<br>**Pimpri Chinchwad College of Engineering & Research**<br>**Ravet, Pune**<br>An Autonomous Institute \| NBA Accredited (4 UG Programs) \|<br>NAAC A++ Accredited \| ISO 21001:2018 Certified<br>**IQAC PCCOER** | |
| **Academic Year:**<br>2026 – 27<br>**Term:** I | **Vision – Mission** | Record No.:<br>**ACAD/R/00** |

Date: 1.7.2026

## Vision – Mission of the Institute

### Vision

To be a globally recognized Institute of technological education and research for the holistic development of aspirants, through excellence in education, innovation and collaborations to fulfill the expectations of all stakeholders..

### Mission

M1: To design and deliver state-of-art knowhow through experiential learning based on changing needs of industry and society worldwide, to ensure the employability and employment of each aspirant.

M2: To enhance the collaborative partnership between Industry and Institute at national and international levels for commercializing and transferring the latest technological know- how towards societal, ethical and economic development.

M3: To achieve and sustain institute position as one of the topmost recognized and ranked institutes in technical and technological education.

## Vision – Mission of the Computer Department

### Vision

To produce globally competent computer engineering professionals through technical Excellence, innovative and research-driven learning, and ethical responsibility, fostering holistic development to meet industry, academic and societal expectations.

### Mission:

1. Establish strong fundamentals, domain knowledge and skills among the students with analytical thinking, conceptual knowledge, social awareness and expertise in the latest tools & technologies to serve industrial demands.
2. Establish leadership skills, team spirit and high ethical values among the students to serve industrial demands and societal needs.
3. Guide students towards Research and Development, and a willingness to learn by connecting themselves to the global society.

**Program Educational Objectives**:

| PO | Program Educational Outcomes |
|---|---|
| **PEO1** | To equip graduates with strong computer engineering fundamentals, practical skills, and ethical responsibility to meet / address global industry, academic, and societal challenges. |
| **PEO2** | To imbibe innovative, research driven and critical thinking practices in graduates to develop creative solutions to real-world problems. |
| **PEO3** | To prepare graduates who will pursue lifelong learning and exhibit leadership, teamwork, and effective communication skills to excel in multidisciplinary environments and contribute to society. |

**Program Specific Outcomes**

| PSO | Program Specific Outcomes |
|---|---|
| **PSO1** | **Problem Solving Skills**: The ability to design, develop, deploy, and evaluate computing systems and applications using open-ended programming environments and suitable engineering tools, technologies, and best practices. |
| **PSO2** | **Professional Skills**: The ability to understand, analyse, design and develop computer Programs in the areas related to algorithms, system programming, web design, data analytics, internet of things, software management, networking and security. |
| **PSO3** | **Successful Career and Entrepreneurship**: The ability to demonstrate professional ethics, effective communication, teamwork, and leadership skills, along with a commitment to lifelong learning, enabling graduates to excel in industry roles, pursue higher education, contribute to entrepreneurial ventures, and generate IPR and high-quality engineering solutions. |

**Program Outcomes**

| PO | Program Outcomes |
|---|---|
| **PO1** | **Engineering knowledge**: Apply knowledge of mathematics,natural science, computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems. |
| **PO2** | **Problem analysis**: Identify, formulate, review research literature And analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. |
| **PO3** | **Design/development of solutions**: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. |
| **PO4** | **Conduct investigations of complex problems**: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, Modelling, analysis & interpretation of data to provide valid conclusions. |
| **PO5** | **Engineering tool usage**: Create, select and apply appropriate techniques, resources and Modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. |
| **PO6** | **The engineer and The World**: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. |
| **PO7** | **Ethics**: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. |
| **PO8** | **Individual and Collaborative Team Work**: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams. |
| **PO9** | **Communication**: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering Cultural, language, and learning differences. |
| **PO10** | **Project Management and Finance**: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary Environments. |
| **PO11** | **Life-long learning**: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest Context of technological change. |

| | | |
|---|---|---|
| | **Pimpri Chinchwad Education Trust's**<br>**Pimpri Chinchwad College of Engineering & Research**<br>**Ravet, Pune**<br>An Autonomous Institute \| NBA Accredited (4 UG Programs) \| NAAC A++ Accredited \| ISO 21001:2018 Certified<br>**IQAC PCCOER** | |
| **Academic Year: 2026 - 27** | **Course Objectives & Course Outcomes** | **Record No.: ACAD/R/08-A**<br>**Term: I** |

**Department: Computer Engineering** **Class – TE**
**Subject Name: Artificial Intelligence Laboratory** **Subject Code: PCC304COM**
**Name of Faculty: Mrs. Priyadarshani Doke (Artificial Intelligence)**

**Course Objectives:**

1. To provide a comprehensive understanding of the fundamental concepts, principles, and methodologies of Artificial Intelligence.
2. To equip students with the ability to design and implement core AI algorithms for problem solving, search, reasoning, and learning.
3. To develop analytical and practical skills for modelling and solving real-world problems using appropriate AI techniques.

**Course Outcomes:**

| CO | Statements | Cognitive level of learning |
|---|---|---|
| **PCC304.1** | Explain and differentiate fundamental concepts, components, and techniques of Artificial Intelligence, including intelligent agents and AI methodologies. | (Understand) |
| **PCC304.2** | Design and implement AI algorithms for state-space search, heuristic search, reasoning, and basic learning tasks using programming tools. | (Apply) |
| **PCC304.3** | Model real-world problems as AI problems and apply appropriate AI techniques to obtain optimal or near-optimal solutions. | (Apply) |
| **PCC304.4** | To apply AI planning techniques for solving classical planning problems and relate them to real-world industrial AI applications. | (Apply) |

**CO-PO Mapping:**

| CO\PO | PO 1 | PO 2 | PO 3 | PO 4 | PO 5 | PO 6 | PO 7 | PO 8 | PO 9 | PO10 | PO11 | PSO 1 | PSO 2 | PSO 3 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **PCC304.1** | 3 | 3 | 2 | – | – | – | 2 | – | – | – | 3 | 3 | 3 | 3 |
| **PCC304.2** | 3 | 3 | 3 | 2 | 3 | – | – | – | – | – | 3 | 3 | 3 | 3 |
| **PCC304.3** | 3 | 3 | 3 | 2 | 3 | – | – | – | – | – | 3 | 3 | 3 | 3 |
| **PCC304.4** | 3 | 3 | 3 | 2 | 2 | 2 | – | – | – | – | 3 | 2 | 3 | 3 |
| **AVERAGE:** | **3** | **3** | **3** | **2** | **3** | **2** | **2** | – | – | – | **3** | **3** | **3** | **3** |

| Savitribai Phule Pune University<br>Third Year of Computer Engineering (2024 Course):<br>PCC304COM : Artificial Intelligence Laboratory | | |
|---|---|---|
| **Teaching Scheme**<br>**Practical: 02 Hours/Week** | **Credit Scheme 01** | **Examination Scheme and Marks**<br>**Term Work: 25 Marks**<br>**Practical: 25 Marks** |
| **Companion Course: Artificial Intelligence** | | |

**Course Objectives:**

- To introduce fundamental concepts of Artificial Intelligence, intelligent agents, and ethical considerations in AI.
- To develop the ability to model and solve problems using state-space search and heuristic algorithms.
- To enable students to design solutions for adversarial and constraint satisfaction problems using appropriate AI techniques.
- To impart knowledge representation skills using propositional and first-order logic with inference mechanisms.
- To equip students with AI planning techniques and relate them to real-world industrial applications.

## Course Outcomes:

On completion of the course, learner will be able to

**Artificial Intelligence**

- **PCC304.1:** Explain fundamental concepts of Artificial Intelligence, Agents and ethical implications of AI in real-world scenarios.
- **PCC304.2:** To formulate real-world problems as state-space search models and implement appropriate heuristic, local, and online search algorithms
- **PCC304.3:** To model real-world competitive and constraint-based problems and implement adversarial search algorithms and CSP techniques.
- **PCC304.4:** To represent real-world knowledge using propositional and first-order logic and apply Inference techniques and derive logical conclusions in knowledge-based AI systems.
- **PCC304.5:** To apply AI planning techniques for solving classical planning problems and Relate them to real-world industrial AI applications.

### Guidelines for Instructor's Manual _

The instructor's manual is to be developed as a reference and hands-on resource. It should include prologue (about University/program/ institute/ department/foreword/ preface), curriculum of the course, conduction and Assessment guidelines, topics under consideration, concept, objectives, outcomes, set of typical applications/assignments/ guidelines, and references.

## Guidelines for Student's Laboratory Journal

The laboratory assignments are to be submitted by student in the form of journal. Journal consists of Certificate, table of contents, and handwritten write-up of each assignment (Title, Date of Completion, Objectives, Problem Statement, Software and Hardware requirements, Assessment grade/marks and assessor's sign, Theory- Concept in brief, algorithm, flowchart, test cases, Test Data Set(if applicable), mathematical model (if applicable), conclusion/analysis). Program codes with sample output of all performed assignments are to be submitted as softcopy. As a conscious effort and little contribution towards Green IT and environment awareness, attaching printed papers as part of write-ups and program listing to journal must be avoided. Use of DVD containing students programs maintained by Laboratory In-charge is highly encouraged. For reference one or two journals may be maintained with program prints in the Laboratory.

## Guidelines for Laboratory /Term Work Assessment

Continuous assessment of laboratory work should be based on overall performance of Laboratory assignments by a student. Each Laboratory assignment assessment will assign grade/marks based on parameters, such as timely completion, performance, innovation, efficient codes, and punctuality.

## Guidelines for Practical Examination

Problem statements must be decided jointly by the internal examiner and external examiner. During practical assessment, maximum weightage should be given to satisfactory implementation of the problem statement. Relevant questions may be asked at the time of evaluation to test the student's understanding of the fundamentals, effective and efficient implementation. This will encourage, transparent evaluation and fair approach, and hence will not create any uncertainty or doubt in the minds of the students. So, adhering to these principles will consummate our team efforts to the promising start of student's academics.

| Practical Assignment No. | Practical Assignment | Content Delivery Methods (CDM) | COs Mapping to the Contents |
|---|---|---|---|
| Part A | | | |
| 1 | Design and implement a simple reflex agent for the Vacuum Cleaner world and analyze its rational behavior in a given environment. | Hand On | **PCC304COM.1** |
| 2 | Implement a model-based intelligent agent capable of navigating a grid environment with obstacles using internal state representation. | Hand On | **PCC304COM.1** |
| 3 | Implement A* Search for 8 puzzle problem | Hand On | **PCC304COM.2** |
| 4 | Implement Tower of Hanoi by State Space Search | Hand On | **PCC304COM.2** |
| Part B | | | |
| 5 | Use BFS to solve a planning problem. Examples can be 8 puzzle problem, Robot path planning, Blocks world etc. | Hand On | **PCC304COM.4** |
| 6 | Implement Backtracking search to solve the 8-Queens constraint satisfaction problem. | Hand On | **PCC304COM.2** |
| 7 | Develop a simple chatbot using predefined rules and pattern matching. | Hand On | **PCC304COM.2** |
| 8 | Develop a game-playing agent using the Minimax algorithm for optimal decision-making. | Hand On | **PCC304COM.3** |
| Part C | | | |
| 9 | Medical Diagnosis System (Rule-Based Expert System) using forward and backward chaining | Hand On | **PCC304COM.4** |
| | Develop a simple rule-based recommendation system (e.g., movie or book suggestion system). | Hand On | **PCC304COM.4** |
| | Any AI based System from Students side | Hand On | **PCC304COM.4** |

# Assignment No: 01

**Title of the Assignment:** Simple reflex agent

**Problem statement:** Design and implement a simple reflex agent for the Vacuum Cleaner world and analyze its rational behavior in a given environment.

**Objective:**

- To understand the concept of Rationality.
- To implement simple reflex agent for Vacuum Cleaner world.

**Theory:**

A **Simple Reflex Agent** selects actions based only on the current percept, ignoring the history of previous percepts. It follows predefined condition-action rules.

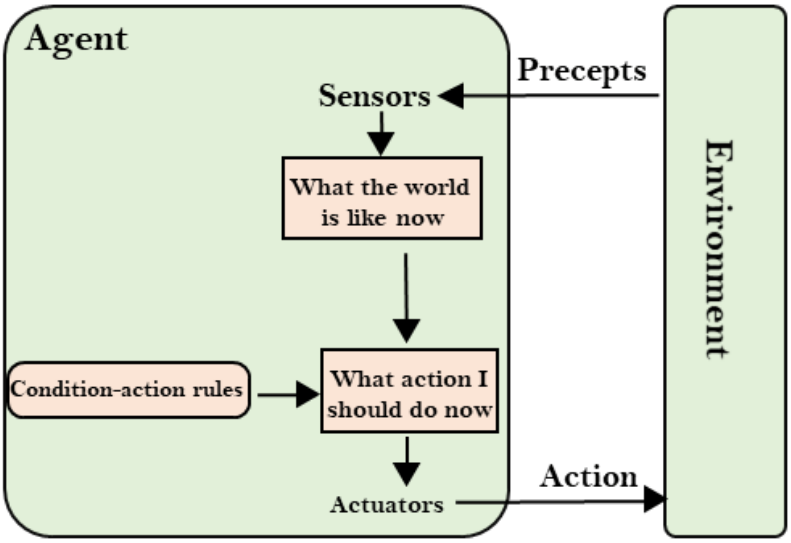


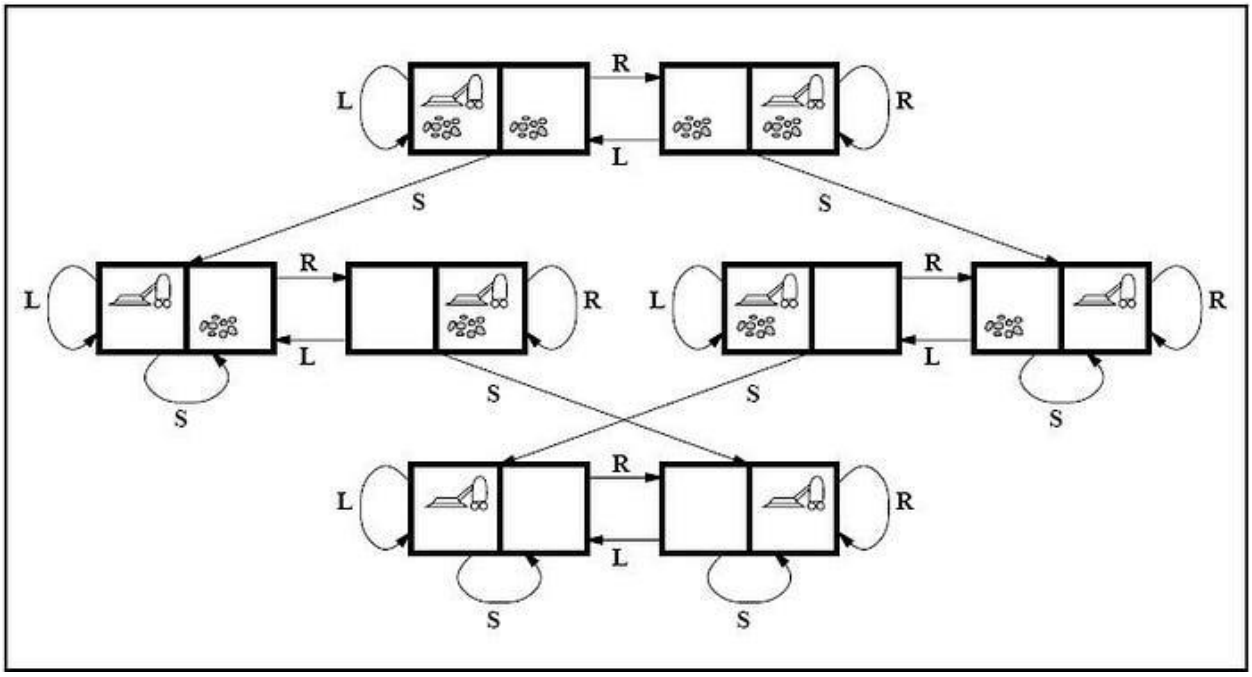

**Figure: Simple Reflex Agent**

These agents do not store historical data or plan ahead. Their decision-making loop is governed by three primary components:

1. **Sensors:** Collect real-time data about the current environment.
2. **Condition-Action Rules:** The agent checks the current input against a pre-programmed rule set (e.g., "If X condition, then Y action").
3. **Actuators:** The mechanism that executes the corresponding action to physically or digitally alter the environment.

In the Vacuum Cleaner World, the agent receives two percepts:

**Scenario and States:**




- **Current location** (A or B)
- **Status of the location** (Dirty or Clean)

Based on these percepts, the agent performs one of the following actions:

- **Suck** – Clean the current location if it is dirty.
- **Move Left** – Move to Room A.
- **Move Right** – Move to Room B.
- **No Operation (NoOp)** – Stop when both rooms are clean.

**Condition-Action Rules**

| Current Location | Status | Action |
|---|---|---|

| Current Location | Status | Action |
|---|---|---|
| A | Dirty | Suck |
| A | Clean | Move Right |
| B | Dirty | Suck |
| B | Clean | Move Left |

**Pseudo code:**

START

Input current location (A/B)
Input current room status (Clean/Dirty)

IF room status is Dirty THEN
Perform SUCK
ELSE
IF location is A THEN
Move RIGHT to B
ELSE
Move LEFT to A
END IF
END IF

STOP

**Algorithm:**

1. Start.

2. Read the current location of the agent.
3. Check whether the current room is Dirty or Clean.
4. If Dirty, perform Suck.
5. Else,
    - If at Room A, move to Room B.
    - If at Room B, move to Room A.
6. Repeat until both rooms are clean.
7. Stop.

**Conclusion:**

We have implemented a simple reflex agent for the Vacuum Cleaner world.

**Oral questions:**

1. What is an Intelligent Agent?
2. What is a Simple Reflex Agent?
3. What are the main components of an agent?
4. What is the Vacuum Cleaner World problem?
5. What are the possible actions of the vacuum cleaner agent?
6. What are the percepts received by the agent?
7. What is meant by the environment in AI?
8. Why is the Vacuum Cleaner World considered a simple AI problem?
9. If Room A is Dirty, Room B is Clean, and the vacuum starts at A, what sequence of actions will it perform?
10. If both rooms are Dirty and the vacuum starts at B, what will be the sequence of actions?
11. If both rooms are already Clean, what should the agent do?
12. What happens if new dirt appears after the rooms have been cleaned?

# Assignment No: 02

**Title of the Assignment**: **Implement a Model-Based Intelligent Agent for Grid Navigation**

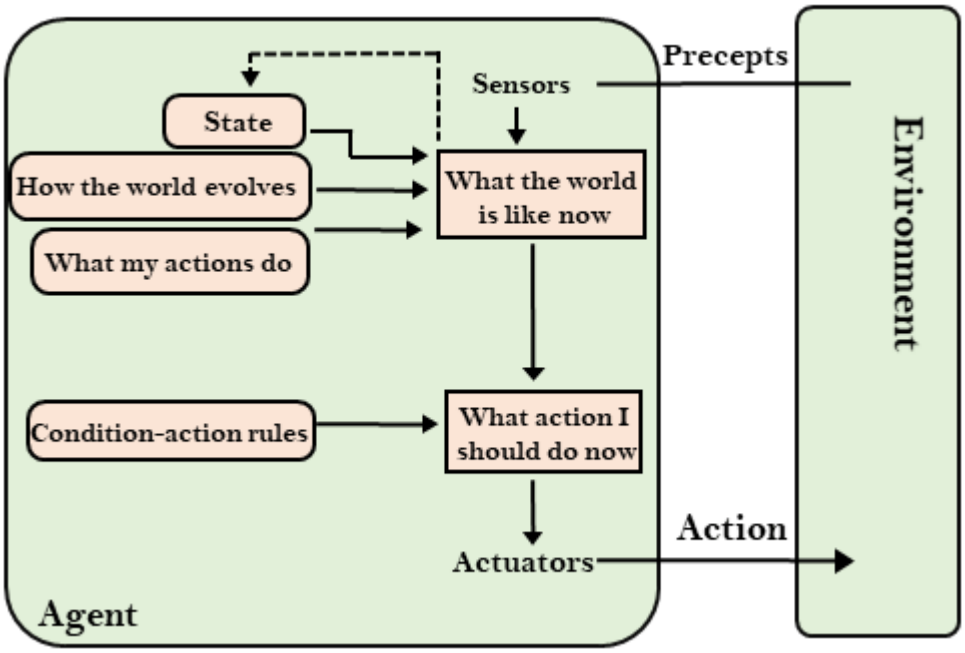
Aim

To implement a **Model-Based Intelligent Agent** capable of navigating a grid environment containing obstacles using an internal state representation.

---

Theory

A **Model-Based Intelligent Agent** maintains an **internal state (memory)** of the environment. Unlike a Simple Reflex Agent, it remembers previous information and uses this memory to make better decisions.




**Key Components of Model-Based Reflex Agent**

1. **Sensors:**

Sensors collect information about the **current state of the environment**. They may be physical (camera, temperature sensor) or virtual (database/API) and provide input for decision-making.

**2. Internal Model:**

The **internal model** stores the agent's knowledge of the environment, including its **state, rules, and action outcomes**. It helps the agent make decisions based on current and past information.

**3. Reasoning Component:**

The **reasoning component** uses sensor data and the internal model to **evaluate the environment, predict outcomes, and select appropriate actions** to achieve the goal.

**4. Actuators:**

**Actuators** execute the selected actions and interact with the environment. They convert the agent's decisions into physical or virtual actions, completing the **perception-action cycle**.

**Condition-Action Rule**

A **Condition-Action Rule** enables a **Model-Based Reflex Agent** to make decisions based on the **current condition** of the environment and its **internal model**. These rules specify the action to perform when a particular condition is met and are usually written as **"If–Then"** statements.

**Examples:**

- **If** an obstacle is detected, **then** stop and change direction.
- **If** the temperature exceeds the threshold, **then** activate the cooling system.

**Ex.**

**In a grid environment:**

- The agent starts from a given position **(S).**
- Some cells contain **obstacles (X)**.
- The agent moves toward the **goal (G)**.
- It stores the cells it has visited in its memory.
- Before moving, it checks whether the next cell is free or blocked.

Here, the internal model consists of two fundamental components that allow the agent to move beyond basic, reactive rules:

1. **Transition Model:** Knowledge of how the world changes. It understands how a specific movement action (e.g., UP, DOWN, LEFT, RIGHT) alters the agent's (x, y) coordinates in the grid.
2. **Sensor Model:** An understanding of how the environment's state is reflected by what the agent can currently perceive (e.g., sensing an obstacle on the next grid cell).

**How It Works**

Instead of only reacting to its immediate surroundings, the agent cycles through five distinct phases:

- **Perceive:** The agent's sensors scan the immediate grid block for obstacles, landmarks, or the target destination.
- **Model the Environment:** The agent updates its internal map with the newly gathered data, keeping track of where it has been and what is around it.
- **Reason & Plan:** The agent calculates the potential consequences of various actions using its internal state. It determines how to progress toward its goal without getting trapped.
- **Act:** The most optimal action is selected and carried out, changing the agent's position on the grid.
- **Update:** The model adjusts its internal parameters based on whether the action successfully achieved the intended spatial result.

---

Algorithm

1. Start.
2. Create a grid with free cells, obstacles, start, and goal.
3. Initialize the agent at the start position.
4. Create an empty **visited** list (internal state).
5. Store the current position in the visited list.
6. If the current position is the goal, stop.
7. Otherwise:
    - Move Right if the next cell is free.
    - Else move Down if the next cell is free.
    - If neither move is possible, display **"No Path Found"**.
8. Repeat until the goal is reached.
9. Display the visited cells.
10. Stop.

---

Sample Output

```
Current Position: (0, 0)
Current Position: (0, 1)
Current Position: (0, 2)
Current Position: (1, 2)
Current Position: (2, 2)
```

Goal Reached!

Visited Cells: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2)]

---

Working Explanation

1. The grid represents the environment.
2. **S** indicates the Start position.
3. **G** indicates the Goal position.
4. **X** represents an obstacle.
5. The agent stores each visited position in the **visited** list, which acts as its internal state.
6. Before moving, the agent checks whether the next cell is blocked.
7. It continues moving until it reaches the goal.

---

Internal State Representation

| Internal State | Purpose |
|---|---|
| Current Position | Stores the agent's present location |
| Visited List | Remembers previously visited cells |
| Grid Map | Stores information about obstacles and the goal |

---

Advantages

- Easy to understand and implement.
- Uses memory to remember visited cells.
- Avoids obstacles.
- Demonstrates the concept of a model-based intelligent agent.
- Suitable for small grid environments.

---

Limitations

- Works only for simple grid layouts.
- Moves only in predefined directions (Right and Down).
- Cannot find the shortest path in complex environments.
- Not suitable for dynamic environments where obstacles change.

---

Applications

- Robot navigation.

- Warehouse automation.
- Game character movement.
- Autonomous mobile robots.
- Indoor service robots.

---

Viva Questions

1. What is a Model-Based Intelligent Agent?
2. What is an internal state?
3. Why is the **visited** list used?
4. What is the difference between a Simple Reflex Agent and a Model-Based Agent?
5. How does the agent avoid obstacles?
6. What are the limitations of this program?
7. What are some real-world applications of model-based agents?

---

Conclusion

The Model-Based Intelligent Agent successfully navigates a simple grid environment using an internal state represented by the **visited** list. It checks for obstacles before moving and remembers previously visited locations, making it more intelligent than a Simple Reflex Agent. This experiment demonstrates the basic concept of memory-based decision-making in artificial intelligence.

# Assignment No: 03

**Title of the Assignment: A* Search for 8 puzzle problem**

**Problem statement:** Implement A star search for 8 puzzle problem.

**Objective:**

- To understand the concept of Informed search techniques.
- To implement A* algorithm for 8 puzzle game problem.

**Theory:**

**A* algorithm:**

A* is a computer algorithm that is widely used in path finding and graph traversal, the process of plotting an efficiently traversable path between multiple points, called nodes.

Noted for its performance and accuracy, it enjoys widespread use.

The key feature of the A* algorithm is that it keeps a track of each visited node which helps in ignoring the nodes that are already visited, saving a huge amount of time. It also has a list that holds all the nodes that are left to be explored and it chooses the most optimal node from this list, thus saving time not exploring unnecessary or less optimal nodes.

Here we use two lists namely ‘open list ‘and ‘closed list‘.The open list contains all the nodes that are being generated and are not existing in the closed list and each node explored after it’s neighboring nodes are discovered is put in the closed list and the neighbors are put in the open list this is how the nodes expand. Each node has a pointer to its parent so that at any given point it can retrace the path to the parent. Initially, the open list holds the start (Initial) node. The next node chosen from the open list is based on its **f score**; the node with the least f score is picked up and explored.

**f-score = h-score + g-score**

A* uses a combination of heuristic value (h-score: how far the goal node is) as well as the g-score (i.e. the number of nodes traversed from the start node to current node).

- A* Algorithm is one of the best path finding algorithms.
- But it does not produce the shortest path always.
- This is because it heavily depends on heuristics.

**Algorithm:**

1. Create a search graph G, consisting solely of the start node, no. Put no on a list called OPEN.

2. Create a list called CLOSED that is initially empty.

3. If OPEN is empty, exit with failure.

4. Select the first node on OPEN, remove it from OPEN, and put it on CLOSED. Called this node n.

5. If n is a goal node, exit successfully with the solution obtained by tracing a path along the pointers from n to no in G. (The pointers define a search tree and are established in Step 7)

6. Expand node n, generating the set M, of its successors that are not already ancestors of n in G. Install these members of M as successors of n in G.

7. Establish a pointer to n from each of those members of M that were not already in G (i.e., not already on either OPEN or CLOSED). Add these members of M to OPEN. For each member, m, of M that was already on OPEN or CLOSED, redirect its pointer to n if the best path to m found so far is through n. For each member of M already on CLOSED, redirect the pointers of each of its descendants in G so that they point backward along the best paths found so far to these descendants.

8. Reorder the list OPEN in order of increasing f values. (Ties among minimal f values are resolved in favor of the deepest node in the search tree.)

9. Go to Step 3.

**8-Puzzle problem:**

In our 8-Puzzle problem, we can define the **h-score** as the number of misplaced tiles by comparing the current state and the goal state or summation of the Manhattan distance between misplaced nodes.

**g-score** will remain as the number of nodes traversed from a start node to get to the current node. We can calculate the **h-score** by comparing the initial (current) state and goal state and counting the number of misplaced tiles.

Given an initial state of a 8-puzzle problem and final state to be reached-



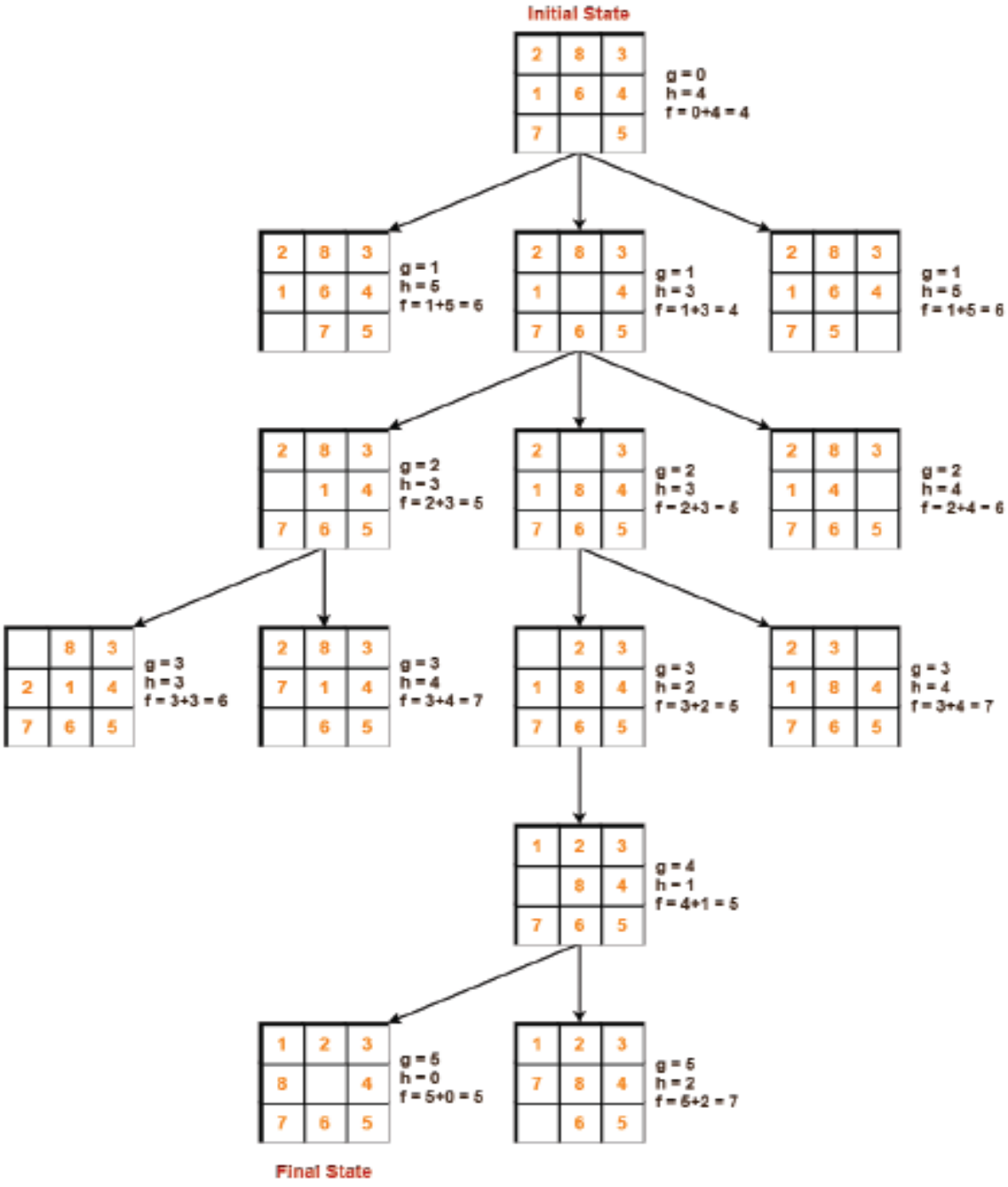

Initial State




Final State

Find the most cost-effective path to reach the final state from initial state using A* Algorithm.

Consider g(n) = Depth of node and h(n) = Number of misplaced tiles.




**Conclusion:**

We have implemented solution for 8 puzzle program using A* search technique.

**Oral questions:**

1. **What are informed and uninformed search techniques?**
2. **List differences between informed and uninformed search techniques?**
3. **What is the evaluation function of A* algorithm?**

4. **What are the applications of A* algorithm?**
5. **What is problem solving agent?**
6. **Is A* algorithm informed or uninformed search technique?**
7. **What is Best first search algorithm?**
8. **Where are searching techniques applied in AI?**
9. **What are Blind search algorithms?**
10. **What is meant by Heuristic?**

# Assignment No: 04

**Title of the Assignment:** Tower of Hanoi by State Space Search

**Problem statement:** To implement a solution for the Tower of Hanoi problem using the Breadth-First Search (BFS) technique within a formalized State Space Search framework.

**Objective:**

- To understand the concept of State Space Search.
- To implement the Tower of Hanoi problem using BFS search technique.

**Theory:**

**State Space Search:**

State space search is an artificial intelligence method used to solve problems by systematically exploring a series of possible states. It models a problem as a graph where nodes are states (situations) and edges are transitions (actions). The goal is to find a path from an initial state to a valid goal state.

**Key Components**

A formal state space search problem consists of six core elements:

1. **Initial State:** The starting point of the search or the agent's current configuration.
2. **Actions:** The set of all legal moves or operations available from a given state.
3. **Transition Model:** The function that defines exactly what new state will result after taking a specific action.
4. **Goal Test:** A condition that checks if a current state is the desired final state.
5. **Path Cost:** The numerical cost (e.g., distance or time) incurred when moving from one state to another.
6. **State Space:** The complete set of all possible states reachable from the initial state.

State Space Search is a general problem-solving framework and does not specify a particular search algorithm. The state space can be explored using different AI search techniques such as Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), Best-First Search, and A* Search. The choice of search algorithm depends on the problem and its implementation.

**Tower of Hanoi problem:**

The Tower of Hanoi is a classic mathematical puzzle involving three rods (A, B, and C) and n disks of different sizes. Initially, all disks are stacked on rod A in decreasing order of diameter - the largest disk at the bottom and the smallest at the top.
Goal is to move the entire stack to another rod (rod C) while following these rules:

- Move only one disk at a time.
- At each step, you can take the top disk from any rod and place it on another rod.
- A disk can only be moved if it is the topmost disk of a rod.
- No larger disk may be placed on top of a smaller disk.

The state space representation of the Tower of Hanoi formally defines the puzzle as a search problem: states capture the positions of all disks, while operators are the legal disk moves. A search algorithm explores this space to find the optimal solution.

**1. Representation of the State Space**

For n disks and 3 pegs, the state space size is exactly $3^n$ because every individual disk can be located on any of the 3 pegs.
If disks are numbered 1 to n (where 1 is the smallest and n the largest), a state can be represented as an array or list of 3 pegs, where each peg holds a list of disks on it.

- **Start State**: All disks on Peg A: (([1, 2, ..., n], [], []))
- **Goal State**: All disks on Peg C: (([], [], [1, 2, ..., n]))

**2. Operators & Constraints**

From any given state, a valid transition (move) depends on two strict rules:

1. You can only move the topmost disk from any peg.
2. You cannot place a larger disk on top of a smaller one.




Figure 1: Initial and Goal States in the Tower of Hanoi

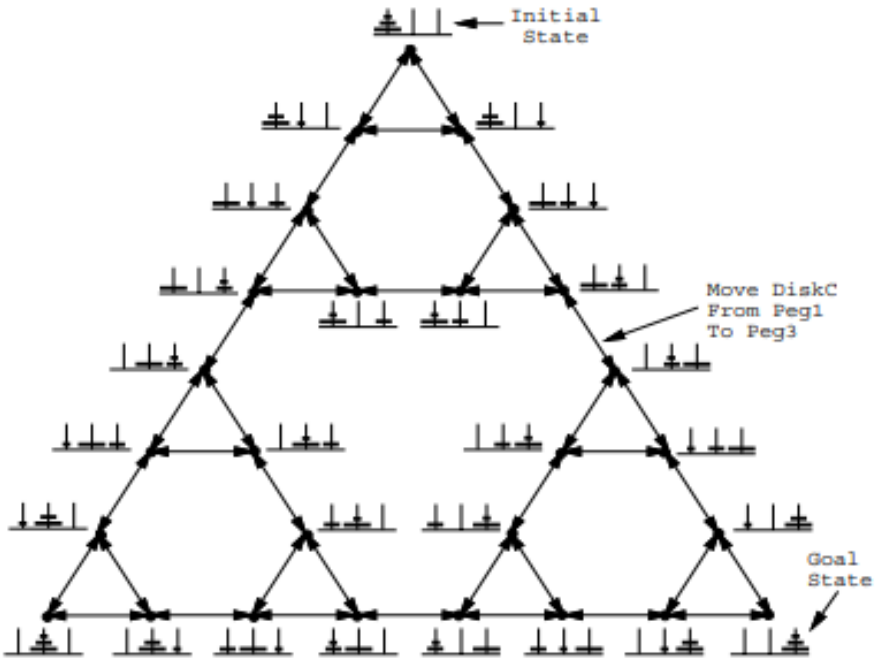



Figure 2: State Space for the Three Disk Tower of Hanoi Puzzle

The Tower of Hanoi problem is modeled as a state-space search problem and solved using Breadth-First Search (BFS). Here, each state represents the position of every disk on the three pegs, and each

valid disk movement generates a new state. BFS explores the state space level by level until the goal state is reached. Since each disk movement has the same cost, BFS guarantees the shortest sequence of moves from the initial state to the goal state.

**Algorithm:**

**Input:** Initial state of all disks and destination peg

**Output:** Minimum sequence of moves to reach the goal state

1. Represent the current arrangement of disks as a state.
2. Insert the initial state into a queue.
3. Mark the initial state as visited.
4. Repeat until the queue becomes empty:
    - Remove the first state from the queue.
    - If it is the goal state, print the solution and stop.
    - Generate all legal moves from the current state.
    - Apply each move to obtain a new state.
    - If the new state has not been visited:
        - Mark it as visited.
        - Insert it into the queue.
5. If no goal state is found, report that no solution exists.

**Characteristics of the Search**

- **State Complexity:** For N disks, there are exactly $3^N$ unique states. For a 3-disk problem, there are $3^3 = 27$ total valid configurations.
- **Time Complexity:** $O(3^N)$ in the worst case, as the BFS algorithm must systematically explore the state space graph to find the goal. The resulting optimal path length is mathematically bounded at $(2^N - 1)$ moves (e.g., 7 moves for 3 disks, 15 moves for 4 disks).
- **Space Complexity:** $O(3^N)$, because an iterative BFS holds the frontier states within a FIFO queue and tracks all encountered states inside a visited set to avoid infinite cyclic loops.

**Conclusion:**

We have implemented a solution for the Tower of Hanoi problem using BFS search technique.

**Oral questions:**

1. What is State Space Search?
2. Why the Tower of Hanoi is considered a state-space search problem?
3. What are the components of a state space (state, initial state, goal state, operators)?
4. Can the Tower of Hanoi be solved using search techniques?
5. What is the goal state in the Tower of Hanoi problem?
6. What are the applications of State Space Search in Artificial Intelligence?
7. Can Tower of Hanoi be solved using recursion? Explain.
8. Why Tower of Hanoi is considered a classic AI problem?
9. How many minimum moves are required to solve the Tower of Hanoi problem for N disks?
10. Why is Breadth-First Search (BFS) used in this implementation?
11. Can the Tower of Hanoi be solved using search techniques other than BFS? Name a few.
12. What is the advantage of BFS over DFS for this problem?

# Assignment No: 05

**Title of the Assignment:** Solving Blocks World Problem using Breadth-First Search (BFS)

**Problem Statement:** Design and implement a Breadth-First Search algorithm to solve the classical **Blocks World** planning problem. Analyze how BFS explores the state space and guarantees the shortest solution sequence.

**Objective:**

- To understand **AI planning techniques**.
- To apply **Breadth-First Search** for solving the Blocks World problem.
- To relate classical planning problems to real-world industrial AI applications.

**Theory**

The **Blocks World problem** is a classical AI planning problem used to demonstrate **state-space search**.

- **Initial State:** Blocks stacked as A on B on C on Table.
- **Goal State:** Blocks stacked as C on B on A on Table.
- **Rules:**
  - A block can move only if it is clear (no block above it).
  - Only one block moves at a time.
  - A block can be placed either on another clear block or on the table.

**Breadth-First Search (BFS):**

- Explores states level by level.
- Uses a queue to store frontier nodes.
- Guarantees the shortest path to the goal.

**Condition-Action Rules**

| **Condition** | **Action** |
|---|---|
| **Block is clear** | Move it to another clear block |
| **Block is clear** | Move it to the table |
| **Goal state reached** | Stop |

**Pseudocode**

Code

START

Put initial state into queue

Mark initial state as visited

WHILE queue not empty:

  Remove front state

  IF state == goal:

    STOP and return solution

  ELSE:

    Generate all valid child states

    FOR each child:

      IF not visited:

        Mark visited

        Add to queue

END

**Algorithm**

1. Start with the initial state.
2. Insert it into the queue and mark visited.
3. While queue is not empty:
    - Remove front state.
    - If it matches goal → stop.
    - Else generate successors.
    - Add unvisited successors to queue.

4. Continue until goal state is reached.
5. Print solution path.

### Output

**Solution Found!**

- **Step 0** Stack 1: [C, B, A]
- **Step 1** Stack 1: [C, B] Stack 2: [A]
- **Step 2** Stack 1: [C] Stack 2: [A] Stack 3: [B]
- **Step 3** Stack 1: [A, B, C]

### Conclusion

We successfully implemented BFS to solve the Blocks World problem. BFS ensures the shortest sequence of moves from the initial state to the goal state. This experiment demonstrates how **search algorithms** can be applied to classical planning problems and extended to more complex real-world scenarios.

### Oral Questions

- What is the **Blocks World problem**?
- Why is BFS suitable for solving planning problems?
- What are the rules for moving blocks?
- How does BFS guarantee the shortest solution?
- What is the difference between BFS and **Depth-First Search** in planning problems?
- How can the Blocks World problem be extended to more blocks?
- What is meant by **state representation** in AI?
- How does BFS avoid revisiting states?

# Assignment No: 06

**Title of the Assignment:** Backtracking Search to Solve the 8-Queens Constraint Satisfaction Problem

**Problem statement:** Implement Backtracking Search to solve the 8-Queens Constraint Satisfaction Problem.

**Objective:**

- To study Constraint Satisfaction Problem using Backtracking.
- To implement 8 queens problem using CSP and Backtracking.

**Theory:**

**Constraint Satisfaction Problem:**

A constraint satisfaction problem (CSP) is defined by a set of variables, x1, x2,….xn , and a set of constraints, c1,c2….cm. Each variable has a nonempty domain of possible values. Each constraint involves some subset of the variables and specifies the allowable combinations of values for that subset. A state of the problem is defined by an assignment of values to some or all of the variables, {xi=vi,xj=vj,…}. An assignment that does not violate any constraints is called a consistent or legal assignment. A complete assignment is one in which every variable is mentioned, and a solution to a CSP is a complete assignment that satisfies all the constraints.

In constraint satisfaction, domains are the spaces where the variables reside, following the problem specific constraints. These are the three main elements of a constraint satisfaction technique. The constraint value consists of a pair of **{scope, rel}**. The **scope** is a tuple of variables which participate in the constraint and **rel** is a relation which includes a list of values which the variables can take to satisfy the constraints of the problem.

Constraint Satisfaction Problems (CSPs) consist of variables, domains, and constraints. The 8-Queens problem requires placing eight queens on an 8×8 chessboard so that no two queens attack each other.

A queen attacks along rows, columns, and diagonals. Backtracking places one queen at a time and backtracks whenever a constraint is violated.

The **8-Queens problem** is a classic CSP where eight queens must be placed on an 8 × 8 chessboard so that no two queens attack each other.

A queen attacks another queen if they are in the:

- Same row
- Same column
- Same principal diagonal
- Same secondary diagonal

Since the algorithm places one queen in each column, it only needs to check:

- Left side of the current row
- Upper-left diagonal
- Lower-left diagonal

**Backtracking Search**

Backtracking is a recursive search algorithm that incrementally builds a solution. If a queen cannot be safely placed, the algorithm removes the previously placed queen and tries another position. This process continues until a valid arrangement is found or all possibilities have been explored.

**Algorithm:**

Input: An empty 8 × 8 chessboard
Output: A valid arrangement of 8 queens

1. Start with the first column.
2. If all columns are filled, print the solution and stop.
3. For each row in the current column:
   a. Check whether placing a queen is safe.
   b. If the position is safe:
      i. Place the queen.

ii. Recursively place a queen in the next column.

iii. If successful, return TRUE.

iv. Otherwise, remove the queen (Backtrack).

4. If no safe position exists in the current column,
   Return FALSE.

**Applications:**

Scheduling, Sudoku, Map Coloring, Timetabling, Resource Allocation, AI Planning, Cryptography.

**Conclusion:**

The Backtracking Search algorithm efficiently solves the 8-Queens CSP by pruning invalid states and recursively exploring only feasible solutions.

**Oral questions:**

What is a CSP?

2. Why is the 8-Queens problem a CSP?
3. What is backtracking?
4. Difference between brute force and backtracking?
5. What constraints are checked?
6. Why only left-side checking?
7. Time complexity?
8. Applications?
9. Number of solutions for 8-Queens?
10. Difference between Backtracking and Branch & Bound?

# Assignment No: 07

**Title of the Assignment**: Develop a Simple Chatbot Using Predefined Rules and Pattern Matching

**Problem Statement**

Design and implement a simple chatbot that responds to user queries using predefined rules and pattern matching techniques.

**Objective**

- To understand the concept of rule-based chatbots.
- To implement a chatbot using predefined rules and pattern matching.
- To learn how AI systems process simple user inputs.
- To analyze the limitations of rule-based conversational agents.

**Theory**

A chatbot is an AI application that interacts with users through text or voice. A rule-based chatbot generates responses using predefined rules or pattern matching. User input is compared with stored patterns; if a match is found, the corresponding response is returned, otherwise a default response is displayed. These chatbots do not learn from previous conversations.

**Working**

1. Accept user input.
2. Convert input to lowercase.
3. Match input with predefined patterns.
4. Display matching response.
5. Display default response if no match.
6. Repeat until user enters bye.

| User Input | Response |
|---|---|
| hello/hi | Hello! How can I help you? |
| how are you | I am fine. Thank you! |
| what is your name | I am an AI Chatbot. |
| help | I can answer simple predefined questions. |
| bye | Goodbye! |
| Others | Sorry, I don't understand your question. |

**Pseudo Code**

START

Display Welcome Message

REPEAT

Input user message

Convert to lowercase

IF hello/hi -> greet

ELSE IF how are you -> fine

ELSE IF what is your name -> AI Chatbot

ELSE IF help -> help message

ELSE IF bye -> Goodbye and EXIT

ELSE -> Sorry, I don't understand.

UNTIL bye

STOP

**Algorithm**

1. Start.
2. Display welcome message.
3. Read user input.
4. Convert input to lowercase.
5. Compare with predefined patterns.
6. Display matching/default response.
7. Repeat until 'bye'.
8. Stop.

**Conclusion**

A simple chatbot was implemented using predefined rules and pattern matching. It responds correctly to known inputs but cannot understand unknown queries or context.

**Oral Questions**

7. What is a chatbot?
8. What is a rule-based chatbot?
9. What is pattern matching?
10. How does a rule-based chatbot work?
11. What are predefined rules?
12. What are the advantages of rule-based chatbots?
13. What are their limitations?

14. Difference between rule-based and AI-based chatbots?
15. What happens if no pattern matches?
16. What is NLP?
17. Can rule-based chatbots learn?
18. Give two applications of chatbots.

# Assignment No: 08

**Title of the Assignment:** Game-playing agent using the Minimax algorithm

**Problem statement:** Develop a game-playing agent using the Minimax algorithm for optimal decision-making. (TIC-TAC-TOE)

**Objective:**

- To understand the concept of game playing agent.
- To implement Minimax algorithm for solving TIC-TAC-TOE game problem.

**Theory:**

**Minimax** (sometimes **minmax**) is a decision rule used in decision theory, game theory, statistics and philosophy for *mini*mizing the possible loss while *max*imizing the potential gain. Alternatively, it can be thought of as maximizing the minimum gain (**maximin**). Originally formulated for two-player zero-sum game theory, covering both the cases where players take alternate moves and those where they make simultaneous moves, it has also been extended to more complex games and to general decision making in the presence of uncertainty.

A **minimax algorithm** is a recursive algorithm for choosing the next move in an n-player game, usually a two-player game. A value is associated with each position or state of the game. This value is computed by means of a position evaluation function and it indicates how good it would be for a player to reach that position. The player then makes the move that maximizes the minimum value of the position resulting from the opponent's possible following moves. If it is **A**'s turn to move, **A** gives a value to each of his legal moves.

A simple version of the minimax *algorithm*, stated below, deals with games such as tic-tac-toe, where each player can win, lose, or draw. If player A *can* win in one move, his best move is that winning move. If player B knows that one move will lead to the situation where player A *can* win in

one move, while another move will lead to the situation where player A can, at best, draw, then player B's best move is the one leading to a draw. Late in the game, it's easy to see what the "best" move is. The Minimax algorithm helps find the best move, by working backwards from the end of the game. At each step it assumes that player A is trying to **maximize** the chances of A winning, while on the next turn player B is trying to **minimize** the chances of A winning (i.e., to maximize B's own chances of winning).

**Tic - Tac – Toe:**

**Tic-tac-toe**, also rendered **wick wack woe** (in some Asian countries), or **noughts and crosses/Xs and Os** as it is known in the UK, Australia and New Zealand, is a pencil-and-paper game for two players, **X** and **O**, who take turns marking the spaces in a 3×3 grid. The **X** player usually goes first. The player who succeeds in placing three respective marks in a horizontal, vertical, or diagonal row wins the game.

The following example game is won by the first player, X:



The simplicity of tic-tac-toe makes it ideal as a pedagogical tool for teaching the concepts of combinatorial game theory and the branch of artificial intelligence that deals with the searching of game trees.

**Algorithm:**

1. Start with the current game state as the root of the game tree.
2. Generate all legal moves for the current player.
3. Recursively expand the game tree, alternating between the maximizing player (MAX) and the minimizing player (MIN).
4. Stop when a terminal state is reached (win, loss, or draw), or at a specified depth if using a cutoff.
5. Assign utility values to terminal states (or evaluation-function values at the cutoff depth).
6. Back up the values through the tree:

   a. At MAX nodes, choose the maximum child value.
   b. At MIN nodes, choose the minimum child value.
7. Select the move from the root that has the highest minimax value.

**Conclusion:**

We have implemented Minimax algorithm for solving TIC-TAC-TOE game problem.

**Oral questions:**

1. What is the Minimax algorithm, and how does it work in Tic-Tac-Toe?
2. Why is Tic-Tac-Toe considered a suitable game for implementing the Minimax algorithm?
3. What is the difference between the maximizing player and the minimizing player in Minimax?
4. How does the evaluation (utility) function determine whether a game state is a win, loss, or draw?
5. What is the time complexity of the Minimax algorithm, and how can it be improved using Alpha-Beta Pruning?
6. What are the limitations of the Minimax algorithm when applied to more complex games like chess?

# References

## Text Books:


1. Stuart Russell and Peter Norvig, "Artificial Intelligence: A Modern Approach", Third edition, Pearson, 2003, ISBN :10: 0136042597
2. Deepak Khemani, "A First Course in Artificial Intelligence", McGraw Hill Education(India), 2013, ISBN : 978-1-25-902998-1
3. Elaine Rich, Kevin Knight and Nair, "Artificial Intelligence", TMH, ISBN-978-0-07-008770-5.


## Reference Books:


1. Nilsson Nils J , "Artificial Intelligence: A new Synthesis", Morgan Kaufmann Publishers Inc. San Francisco, CA, ISBN: 978-1-55-860467-4
2. Patrick Henry Winston, "Artificial Intelligence", Addison-Wesley Publishing Company, ISBN: 0- 201-53377-4
3. Andries P. Engelbrecht, "Computational Intelligence: An Introduction", 2nd Edition-Wiley IndiaISBN: 978-0-470-51250-0
4. Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Python" John Wiley & Sons, Incorporated, 2013ISBN1118476735, 9781118476734.
5. Robert Lafore "Data Structures and Algorithms in Java", Second Edition Copyright © 2003 by Sams Publishing.
6. Adam Drozdek "Data Structures and Algorithms in C++",ISBN-13: 978-1-133-60842-4 ISBN-10: 1-133-60842-6 Cengage Learning.
7. Building-Chatbots-with-Python-Using-Natural-Language-Processing-and-Machine-Learning-by-Sumit-Raj, ISBN No.:978-1484240953.